

RENSSELAER MECHATRONICS

Blinking the LED

Lab Objectives

- Install and verify Arduino software package for Simulink using a digital output to light a LED
- Communicate with the target board (Arduino) using external mode by changing the brightness of an LED with PWM
- Use overrun detection to determine how fast you can run your code

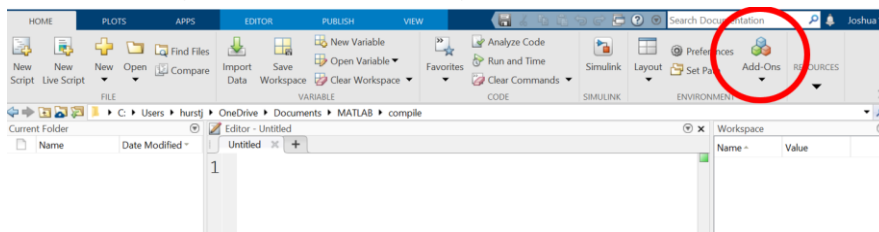
Part 1: Arduino Toolbox Installation for Simulink

Objective:

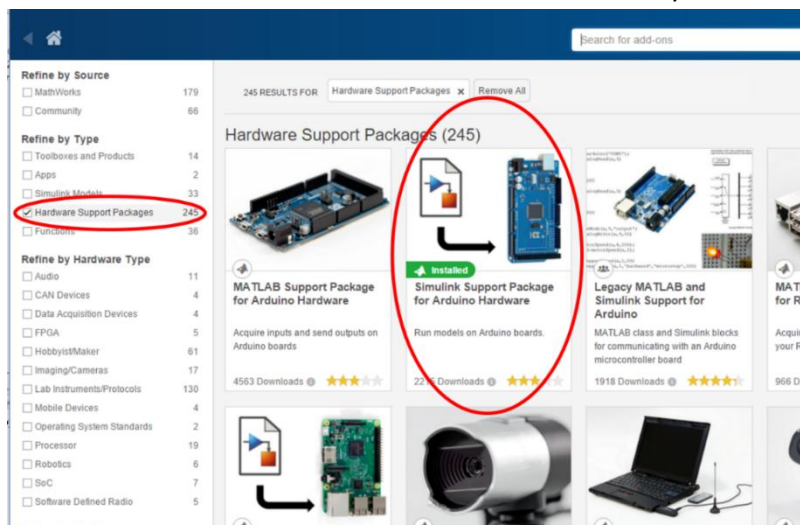
- Install and verify Arduino software package for Simulink using a digital output to light a LED with MATLAB/Simulink 2018b or later.

Simulink Arduino Library Installation:

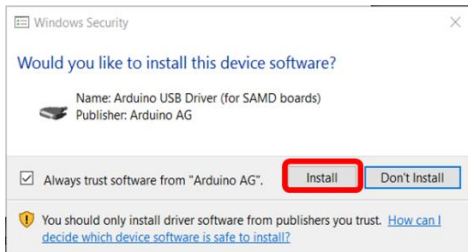
- Open MATLAB by right clicking the icon and selecting “Run as Administrator”
- Click Add-Ons (under the “HOME” tab):



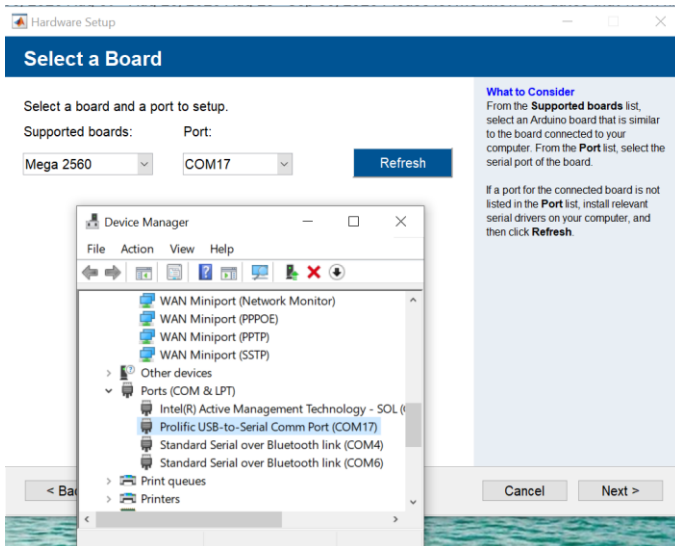
- The add on explorer window will appear. Under “Filter by type” select “Hardware Support Packages” then “Simulink Support for Arduino Hardware”: (you can also type “Simulink Support for Arduino Hardware” in the ‘search for addons’ box if you have trouble finding it.



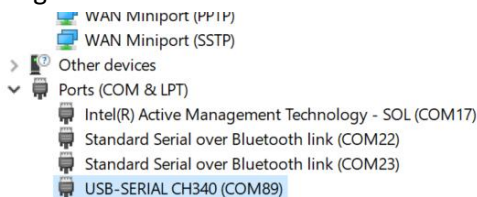
- When prompted, log in using your MathWorks credentials. If you do not have an account you will need to create one.
- Follow the onscreen prompts to install the library.
- During this process, install the drivers if prompted:



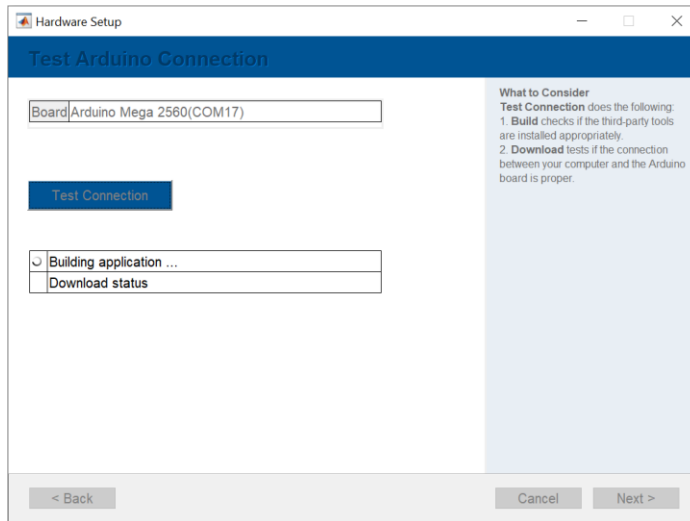
- Once everything is installed, choose setup hardware now in the prompted window. During Hardware Setup, it might not recognize your board. In this case, manually select the board; use Device Manager to find the COM port for your board and click next.



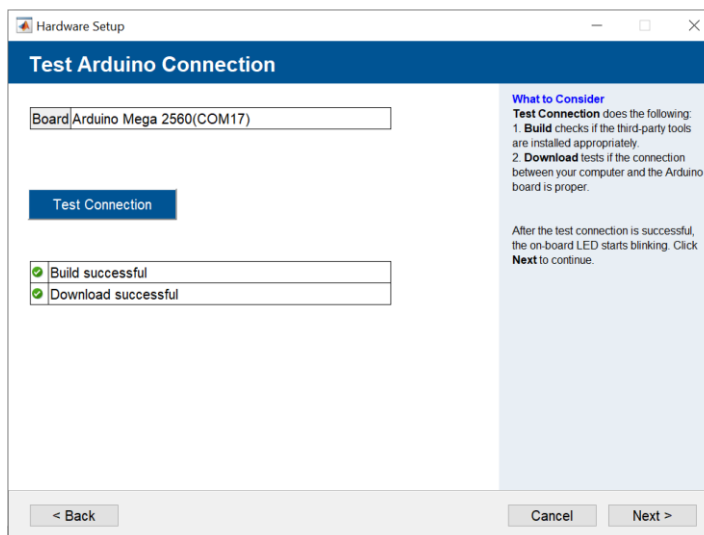
It might read “Prolific USB-to-Serial Comm Port” Or “USB-SERIAL CH340”.



- Click Verify Setup to make sure it can communicate with your board



- It should indicate that it has downloaded test code that will make Led 13 blink on your board



- Click Next, then Finish.

How do I find the COM port for my Arduino Board:

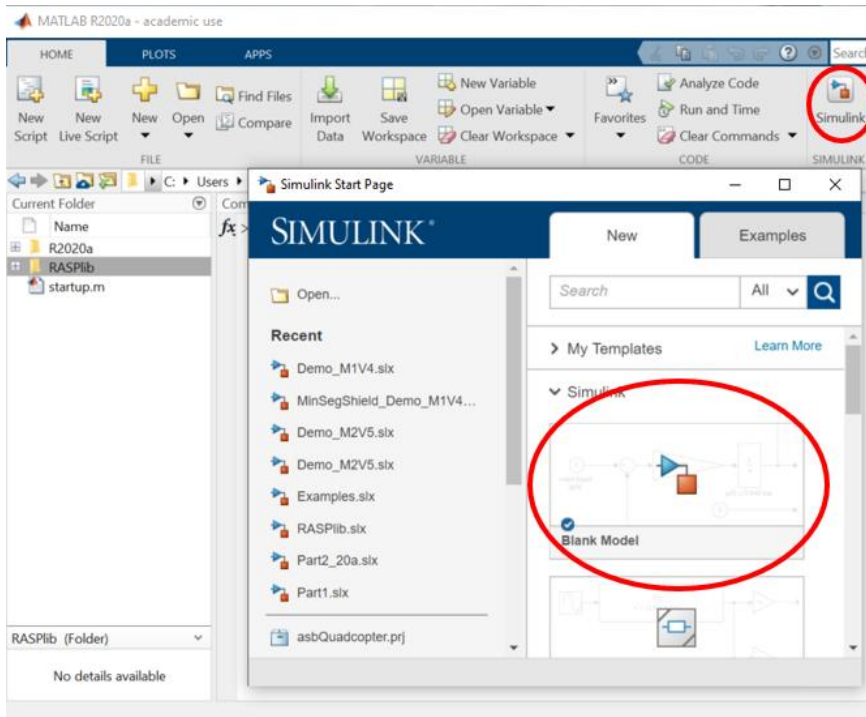
- Go to Device Manager and look for your Arduino Mega board (Arduino Mega 2560). Your Arduino board will be under Ports (COM & LPT).
- It may appear as “Arduino Mega”, “Prolific USB-to-Serial Comm..”, or “USB-SERIAL CH340” depending on your board. With the device manager open, unplug and plug in the USB cable to find which device appears.

Simulink Setup:

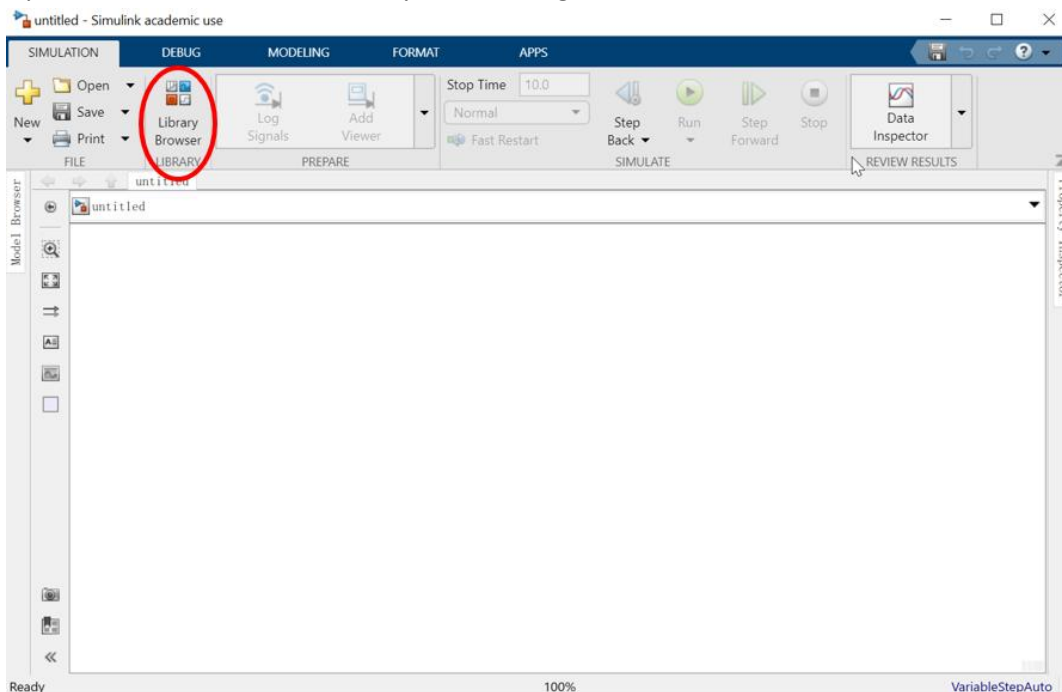
In order to ensure that your computer can properly communicate with the Arduino board, build and run the following Simulink diagram using the steps below:



- 1) Open MATLAB then open a new Simulink model by clicking the Simulink icon at the top of the screen, then the “Blank Model” in the new window:



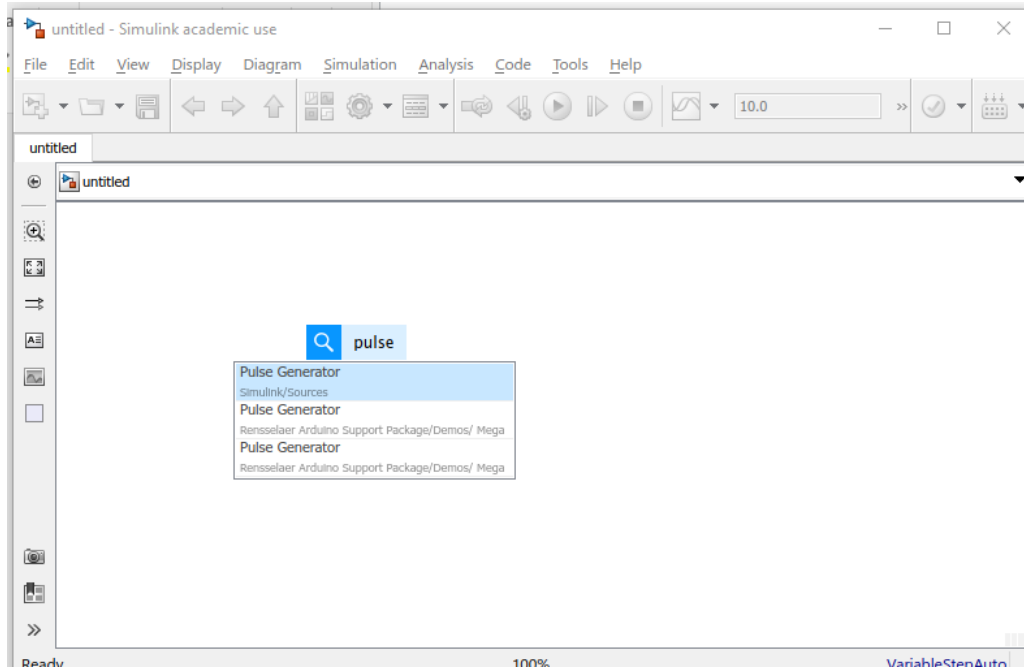
Open the Simulink browser from your new diagram:



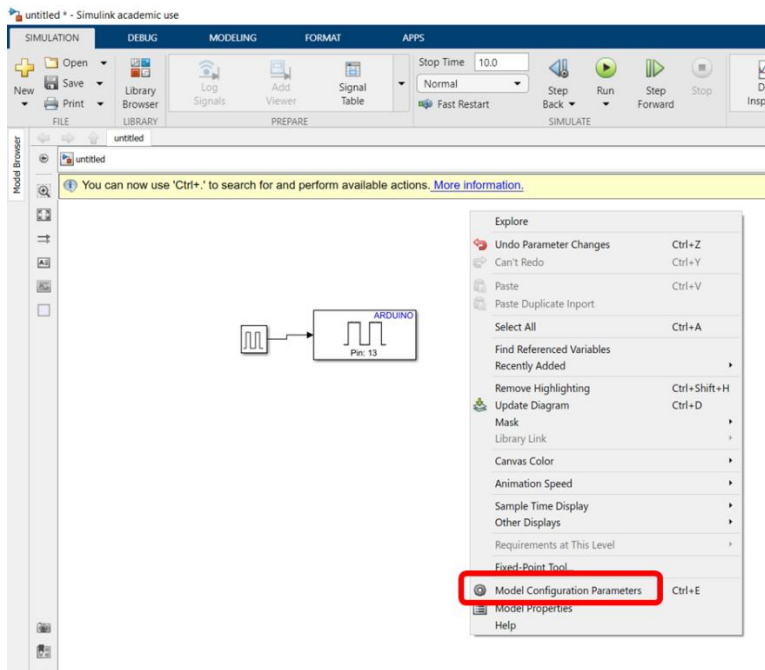
2) Find and assemble the required blocks.

- Pulse Generator is under View->Library Browser->Simulink->Sources
- Digital Output is under View->Library Browser->Simulink Support Package for Arduino Hardware->Common

You can also search for the blocks by starting to type in the empty diagram for the block your looking for:

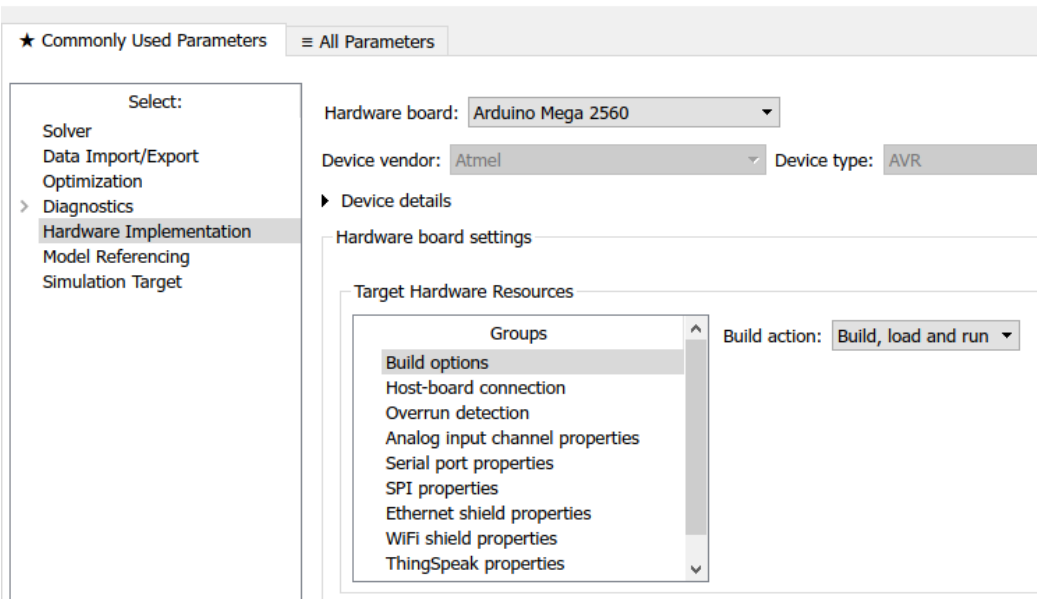


- 3) Change the output pin to 13, which is connected to the onboard LED. This is done by double clicking on the Digital Output block.
- 4) Double click on the Pulse Generator to change the amplitude to 1, the Period to 10, and the pulse width to 50%.
- 5) Right click in the Simulink model background to open Model Configuration Parameters:



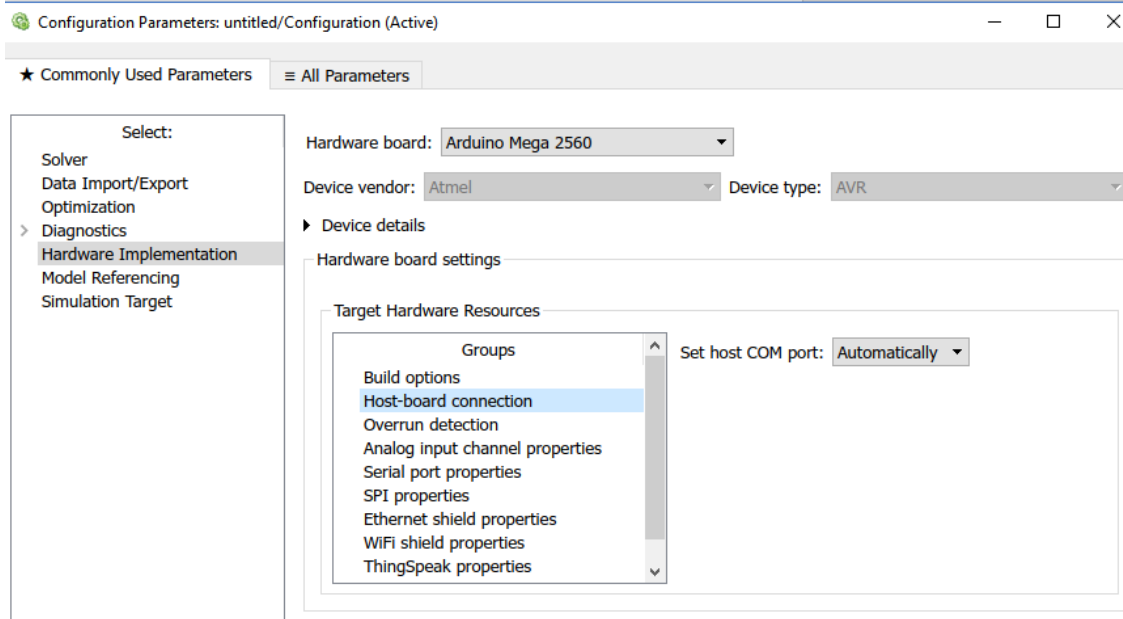
- From the left-hand menu, pick “Hardware Implementation”. Select “Arduino Mega 2560” from the drop-down menu by “Hardware board”.

Configuration Parameters: untitled/Configuration (Active)

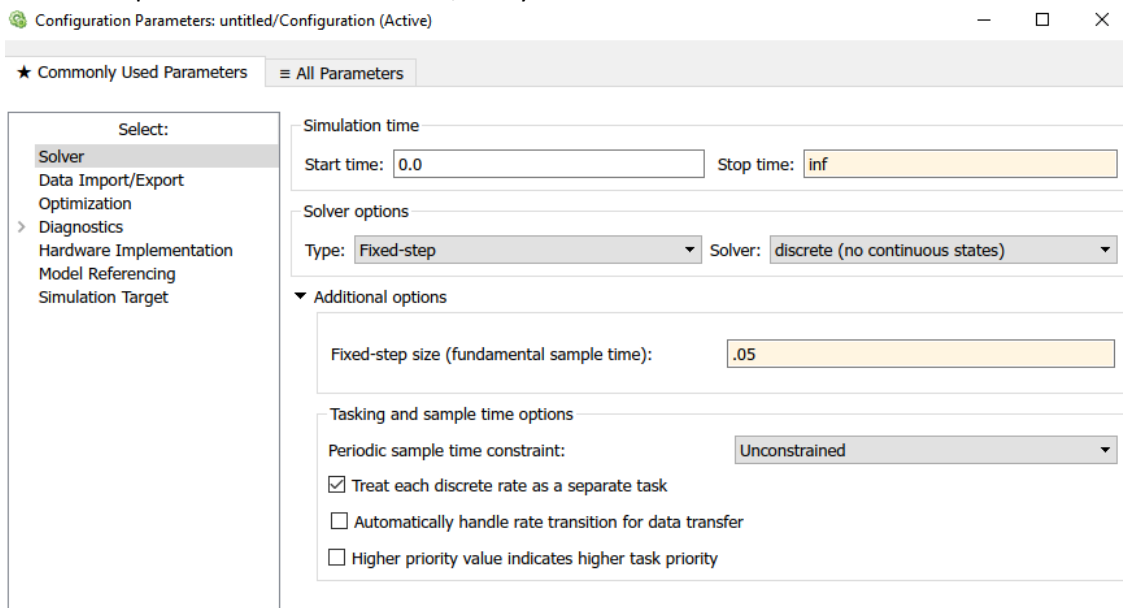


- Under “Target Hardware Resources” and “Host-board connection”, set host COM port: should be set to “Automatically”, click OK. You should specify the COM port “Manually” here if your

board was “Prolific USB-to-Serial Comm..”, or “USB-SERIAL CH340” instead of “Arduino Mega”.

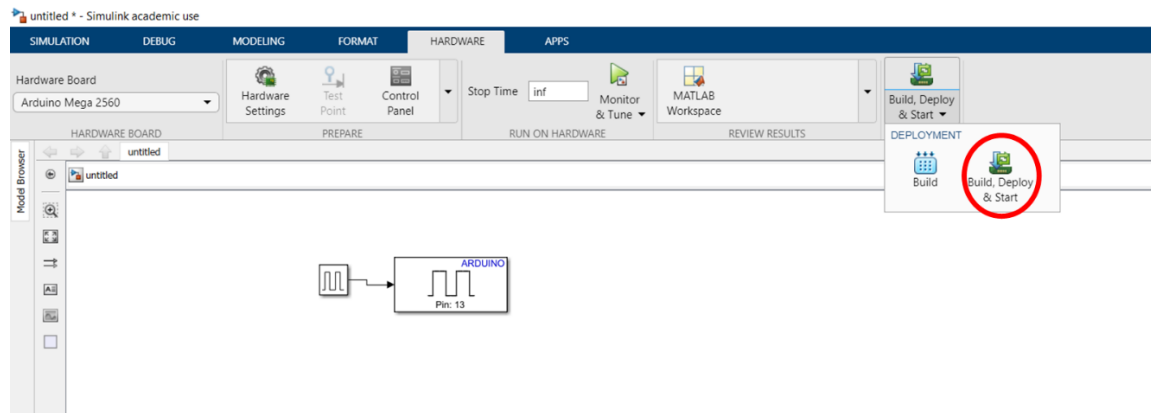


- Go to ‘Solver’ on the left hand menu and type ‘inf’ into the ‘Stop time’ spot so that the model will keep running. Make sure the solver options are set to “Fixed-step” and “discrete (no continuous states)”. The “Fixed-step size” should be 0.05. This is how fast in seconds the control loop will execute. In this case, every 50 milliseconds it will execute the Simulink code.



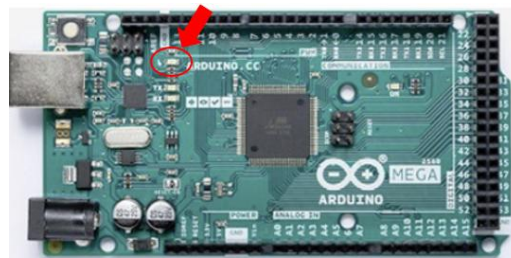
6) Compile and download the code to the board:

- Under the “HARDWARE” tab, choose “Build, Deploy, & Start”, or the keystroke Ctrl+B:



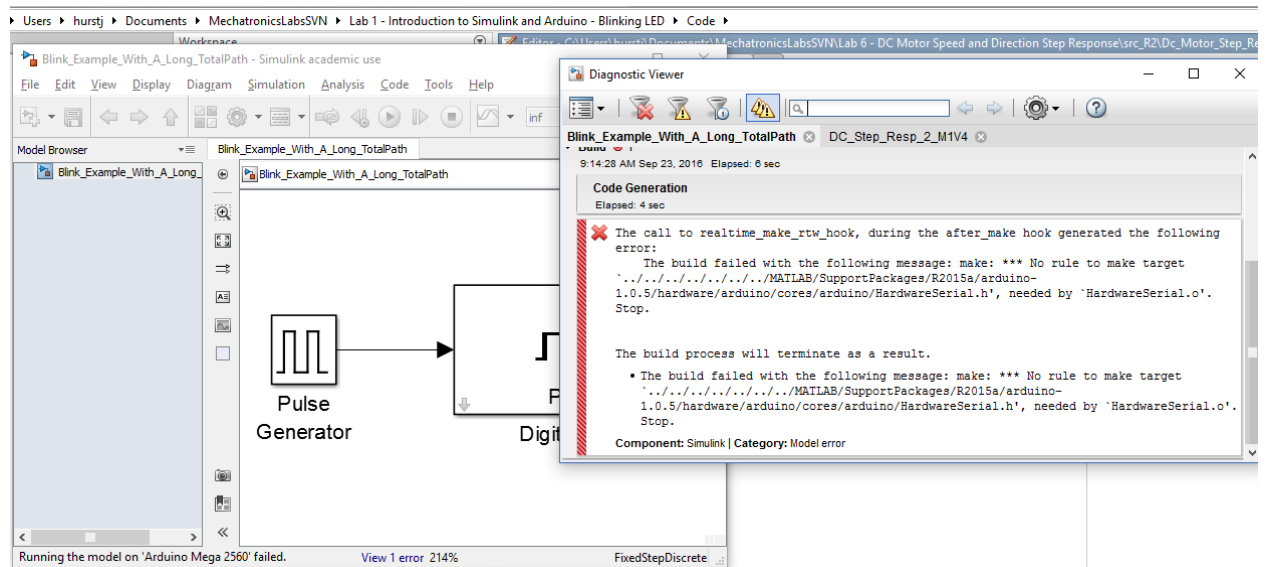
This will download the code to the board and **will not interact or connect with it**. In this “Normal” or “Deploy” Mode the code is downloaded to the board, and no information is passed between it and your computer unless your program specifically instructs the board to do so.

At this point the onboard LED connected to pin 13 should start blinking on and off every 5 seconds as prescribed by the pulse generator block. This LED is on the lower board towards the USB connector. If not, check the troubleshooting section below.



Troubleshooting

- If an error occurs indicating that there is no driver, follow the directions on the screen.
- If MATLAB cannot find the board find COM port number from the device manager and in ‘Configuration Parameters’ set the port manually to the correct number.
 - Also, try unplugging the Arduino USB cable and plugging it back in.
 - Can also try restarting MATLAB
- Some errors appear on the MATLAB workspace screen - if something is not working, check here – the error messages will indicate the problem
- If you get a code generation error like below:

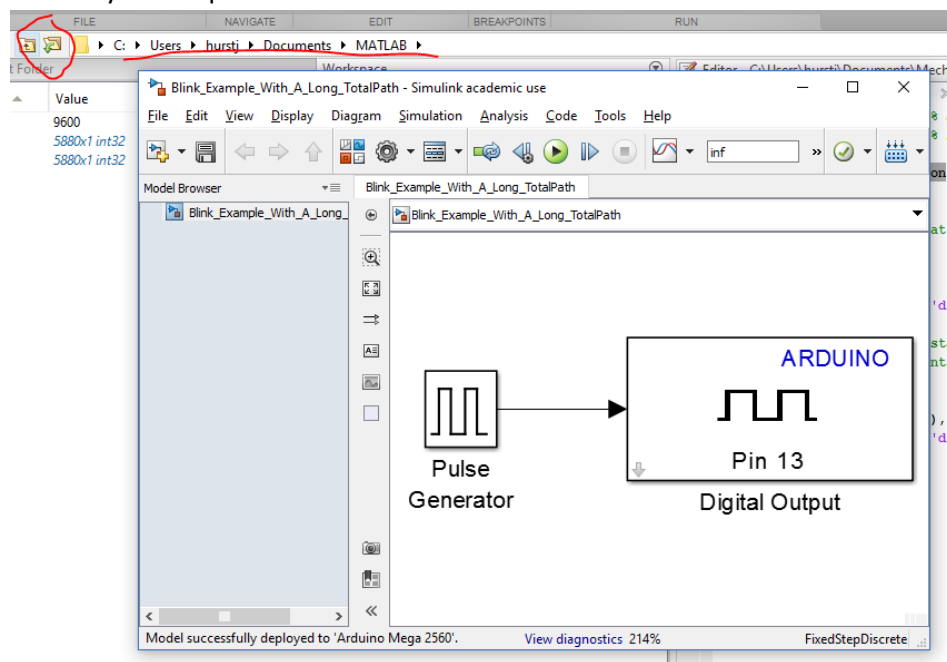


This is because MATLAB is compiling in a directory where the directory and filename is too long.

In the example above the compilation path is:

C:\Users\hurstj\Documents\MechatronicsLabsSVN\Lab 1 - Introduction to Simulink and Arduino - Blinking LED\Code\Blink_Example_With_A_Long_TotalPath.slx

To remedy this you can try a shorter file name, or simply change (point) MATLAB to a shorter directory to compile in like:



It is the same file, but MATLAB will now compile everything in

C:\Users\hurstj\Documents\MATLAB so the compiler won't have a path problem.

- If your file and your compile directory are on a different drive for example D: instead of the installation drive for MATLAB, you also might get a compile error. In this case you will want to

try and move the file and the compile folder to the MyDocuments/MATLAB folder that is created when MATLAB installs.

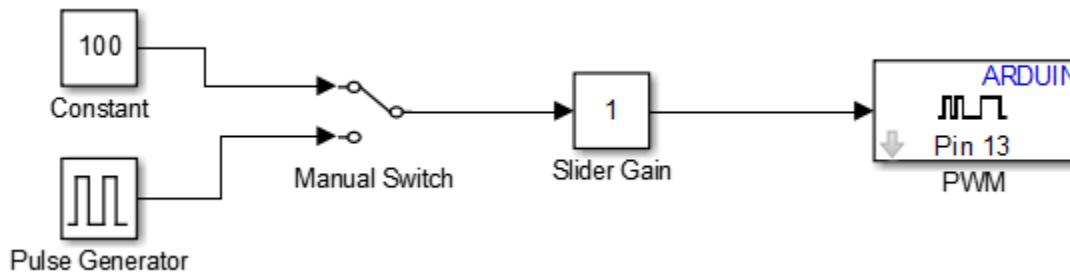
Part 2: Communicating with External Mode

Objective

- Communicate with the target board (Arduino) using external mode by changing the brightness of an LED with PWM

Simulink

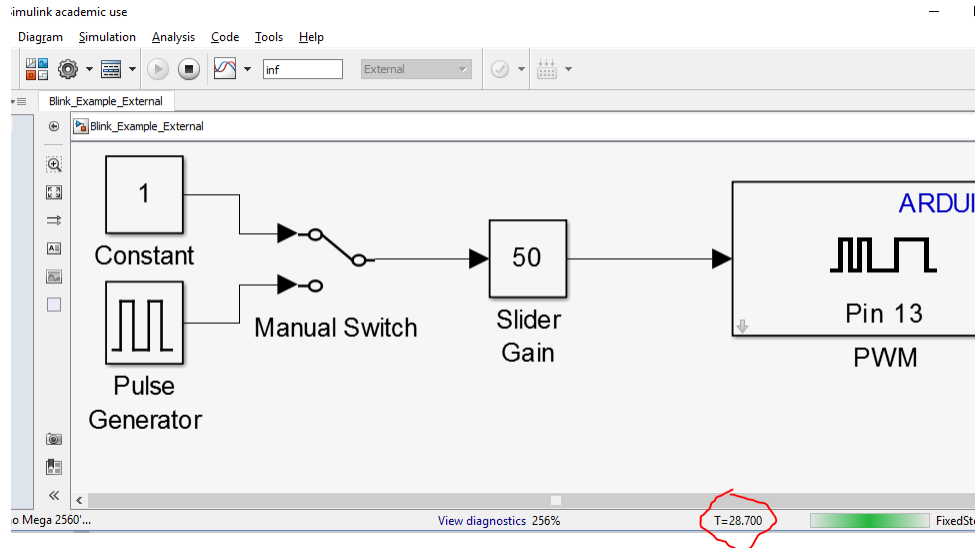
Modify your Simulink diagram to the following



- Pin 13 can also be set as a PWM instead of just an on/off digital output – replace the Digital Output block with the PWM block
- The “Slider Gain” can be found in View->Library Browser->Simulink-> Math Operations. Change the high value to 255 (the max value of the PWM)
- The “Manual Switch” can be found in View->Library Browser->Simulink ->Signal Routing
- The “Constant” can be found in View->Library Browser->Simulink -> “Commonly Used Blocks”

External Mode

- External mode**, enabled by using the “Monitor & Tune” button, **allows bi-directional communication** to/from the application board to the PC, but can limit how fast your code can execute.
 - The “**Deploy, Build, & Start**” button downloads the code and **does not connect or interact with it**, it can allow your code to run faster
 - When external mode is selected it downloads additional code to the board to allow this communication. This does increase the program size (requires more memory)
- When you use external mode, you can change parameters while the system is running
 - However, this has an impact on the performance due to the communication bandwidth.
 - You can tell how fast you can run your code by examining the running clock in the Simulink diagram after you press the play button. When the code is running the green bar will be flashing and the time in seconds should be advancing:

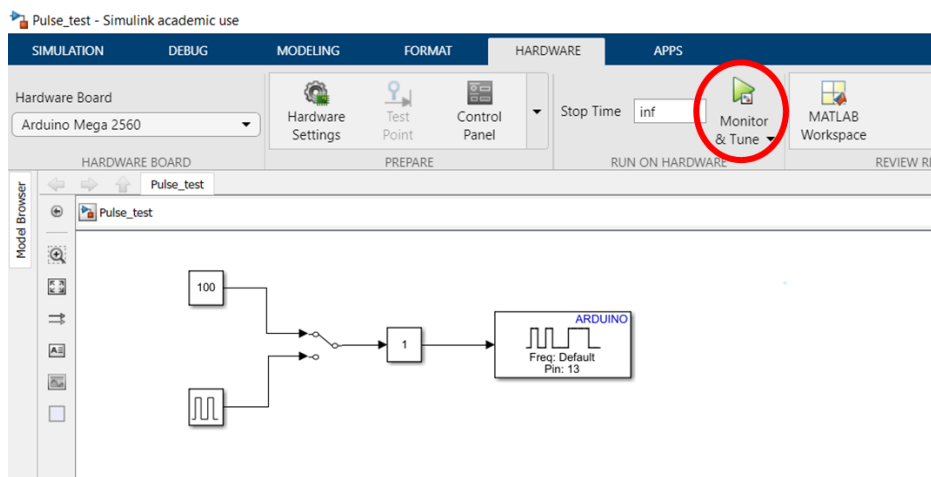


If this clock advances as fast as the normal real world clock your code is “keeping up”. If, for example, it can take 1 minute for the this timer to reach 10 seconds your code is not keeping up with real time and your requested sample time is to fast – it can not compute everything it needs to do fast enough.

- The values change on the Slider Gain and the Manual Switch can be changed while the program is running.
- If necessary, the code can be run faster if external mode is not used, by using “Deploy, Build, & Start” button instead of the “Monitor & Tune” button.

Enabling External Mode:

- Under the “HARDWARE” tab, select “Monitor & Tune”:



Run the code and experiment with the setup. Observe the effects on the LED when changing the value of the Slider Gain, Manual Switch, and the Pulse Generator.

- While running the simulation, you can change the position of the switch by double clicking on the switch

- You can change the gain of the Slider Gain while running the simulation by double clicking it.
 - Adjusting the Pulse Generator, experiment with all three parameters. Which increases brightness? What is the approximate range over which you can observe a difference?
- [Note: These parameters will be addressed in a later lab on PWM]

Stopping the Simulation:

When you are running the simulation in external mode you are connected to the device and are sending data and receiving data.

- Use the green “Monitor & Tune” button to download and run the code in external mode
- Use the square “Stop” button to stop. Since the board is no longer connected, it will usually stop, but sometimes will continue doing the last action before it was disconnected.

Checkpoint:

To complete Lab 1 you will need to show the ability to control the blink of the LED in one program with both the ‘manual switch’ and the ‘slider gain’.

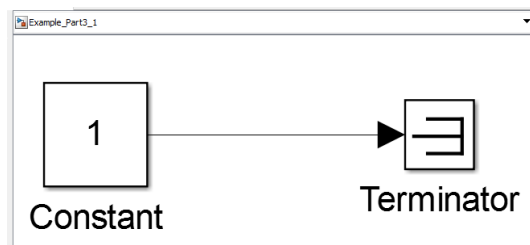
Part 3: Evaluating code execution speed – how fast can I run my code

Objective:

- Determine how fast your code can be run on the target hardware using “Enable overrun detection” in Simulink.
- This can be used in both External and Normal mode to ensure your code is executing in time.

Simulink:

Build the following model:



Note that this code essentially does nothing. The terminator is also in the ‘commonly used blocks’. To determine how fast the hardware can execute a simple loop, use “Enable overrun detection” on pin 13:

Right click in the Simulink model background to open Model Configuration Parameters. In the window that pops up, select “Hardware Implementation” from the left-hand menu, and expand “Hardware board settings”. You will need to select Arduino Mega 2560 in the drop down menu at the top. Under

Target Hardware Resources and Overrun detection, check “Enable overrun detection” and set the pin to 13.

Configuration Parameters: Blink_Example_External/Run on Hardware Configuration (Active)

★ Commonly Used Parameters ≡ All Parameters

Select:

- Solver
- Data Import/Export
- Optimization
- > Diagnostics
- Hardware Implementation**
- Model Referencing
- Simulation Target

Hardware board: **Arduino Mega 2560**

Device vendor: **Atmel** Device type: **AVR**

▶ Device details

Hardware board settings

Target Hardware Resources

Groups

- Build options
- Host-board connection
- Overrun detection**
- Analog input channel properties
- Serial port properties
- SPI properties
- Ethernet shield properties
- WiFi shield properties
- ThingSpeak properties

☒ Enable overrun detection

Digital output to set on overrun: **13**

If the hardware cannot execute your code in the same time you specify the digital output, in this case the LED on pin 13 will light up indicating that you code cannot be executed in the time you have specified. You can use any digital output, but pin 13 is conveniently connected to an LED on our board. For the example code above, start with a sample time of 0.01 milliseconds (.00001 seconds). Download the code with the deploy to hardware button (Normal mode, not external mode) and observe if LED 13 on the board lights up after downloading:

Configuration Parameters: Blink_Example_External/Run on Hardware Configuration (Active) — □ ×

★ Commonly Used Parameters ≡ All Parameters

Select:

- Solver**
- Data Import/Export
- Optimization
- > Diagnostics
- Hardware Implementation
- Model Referencing
- Simulation Target

Simulation time

Start time: **0.0** Stop time: **inf**

Solver options

Type: **Fixed-step** Solver: **discrete (no continuous states)**

▼ Additional options

Fixed-step size (fundamental sample time): **.00001**

Tasking and sample time options

Periodic sample time constraint: **Unconstrained**

☒ Treat each discrete rate as a separate task

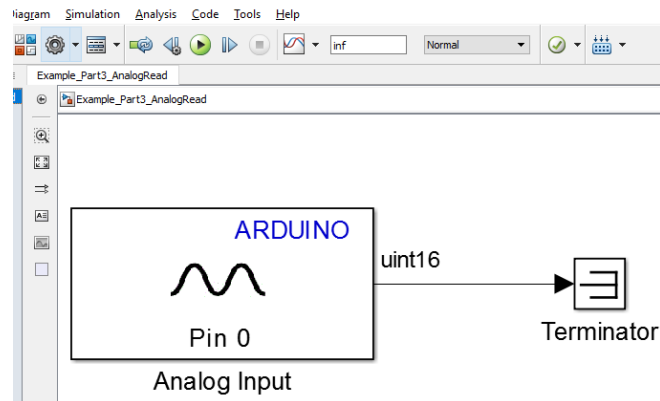
☐ Automatically handle rate transition for data transfer

☐ Higher priority value indicates higher task priority

You should notice LED 13 will light up. This indicates the microprocessor can not run the code every .00001 second, so you have to specify a larger sample time

Keep increasing the sample time (try .02 milliseconds, etc.) until LED 13 does not light up after downloading. This is the fastest you the board can execute your code.

Using this procedure try and see how fast you can run the following, start at a sample time of 0.01 milliseconds.



In this fashion, you can determine how fast your code, or any part of your code takes to run on your hardware.

Questions:

How can you determine how fast you can run code in External mode (Monitor and Tune)?

How can you determine how fast your code can run in Normal mode (Deploy-Build-Start)?

What is the difference between Normal mode and External mode?